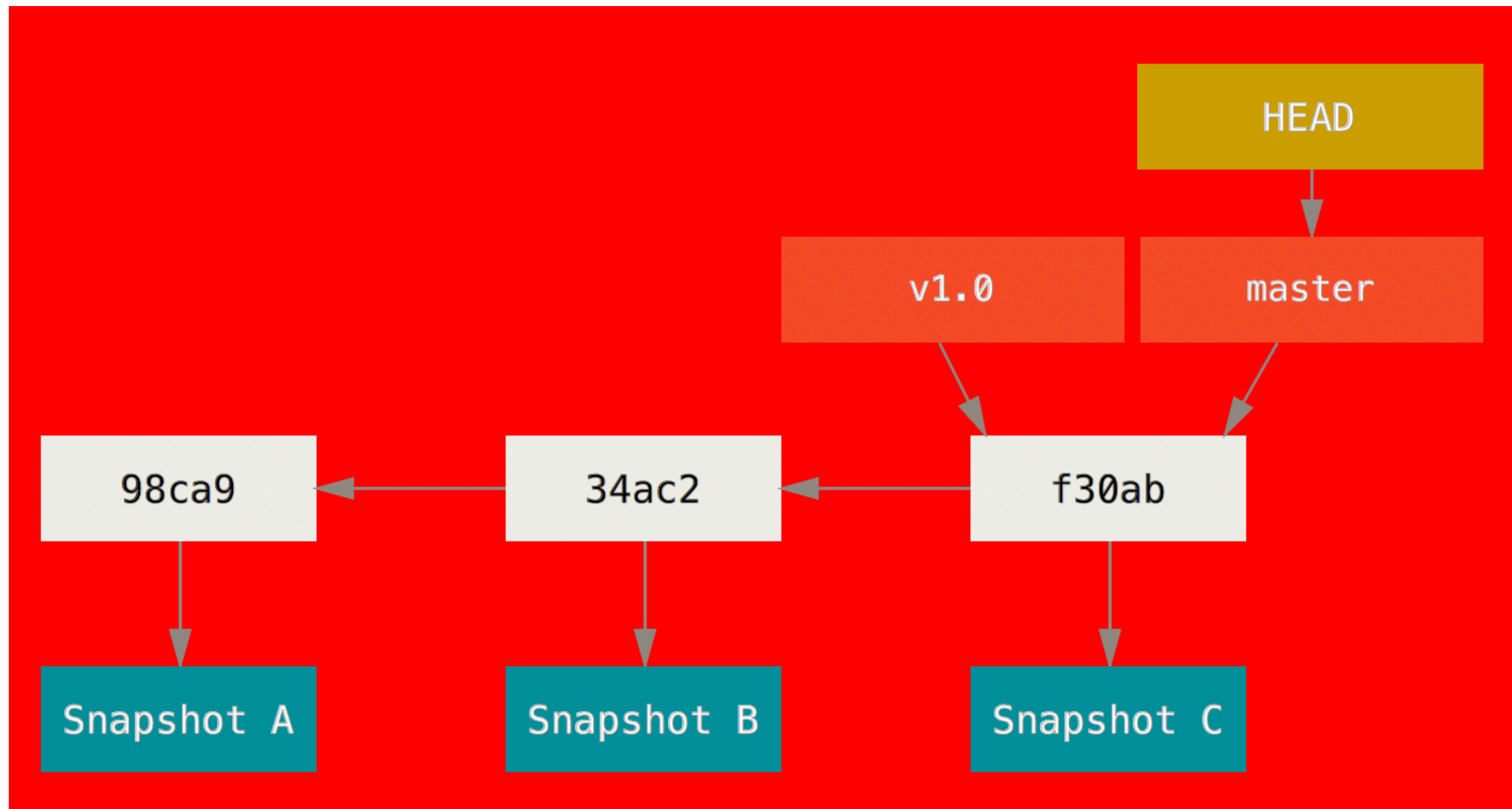
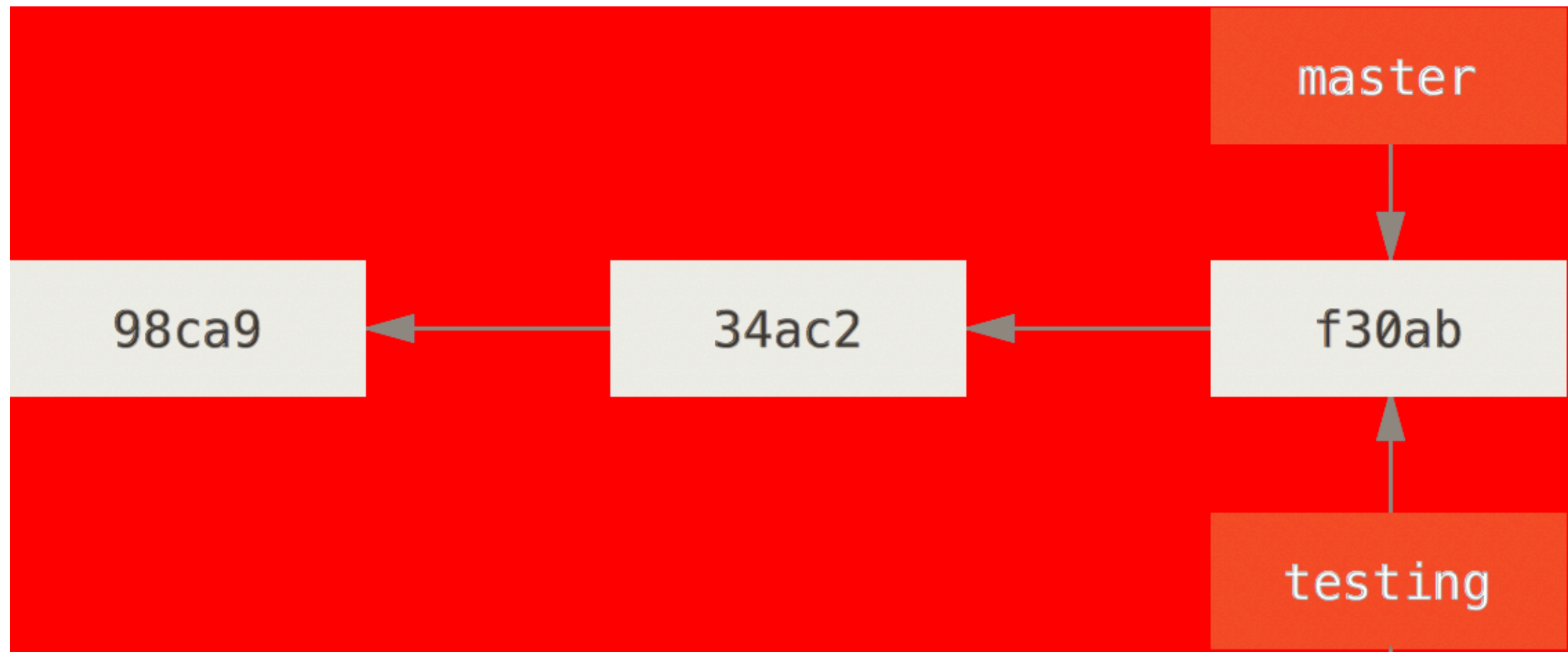


GIT

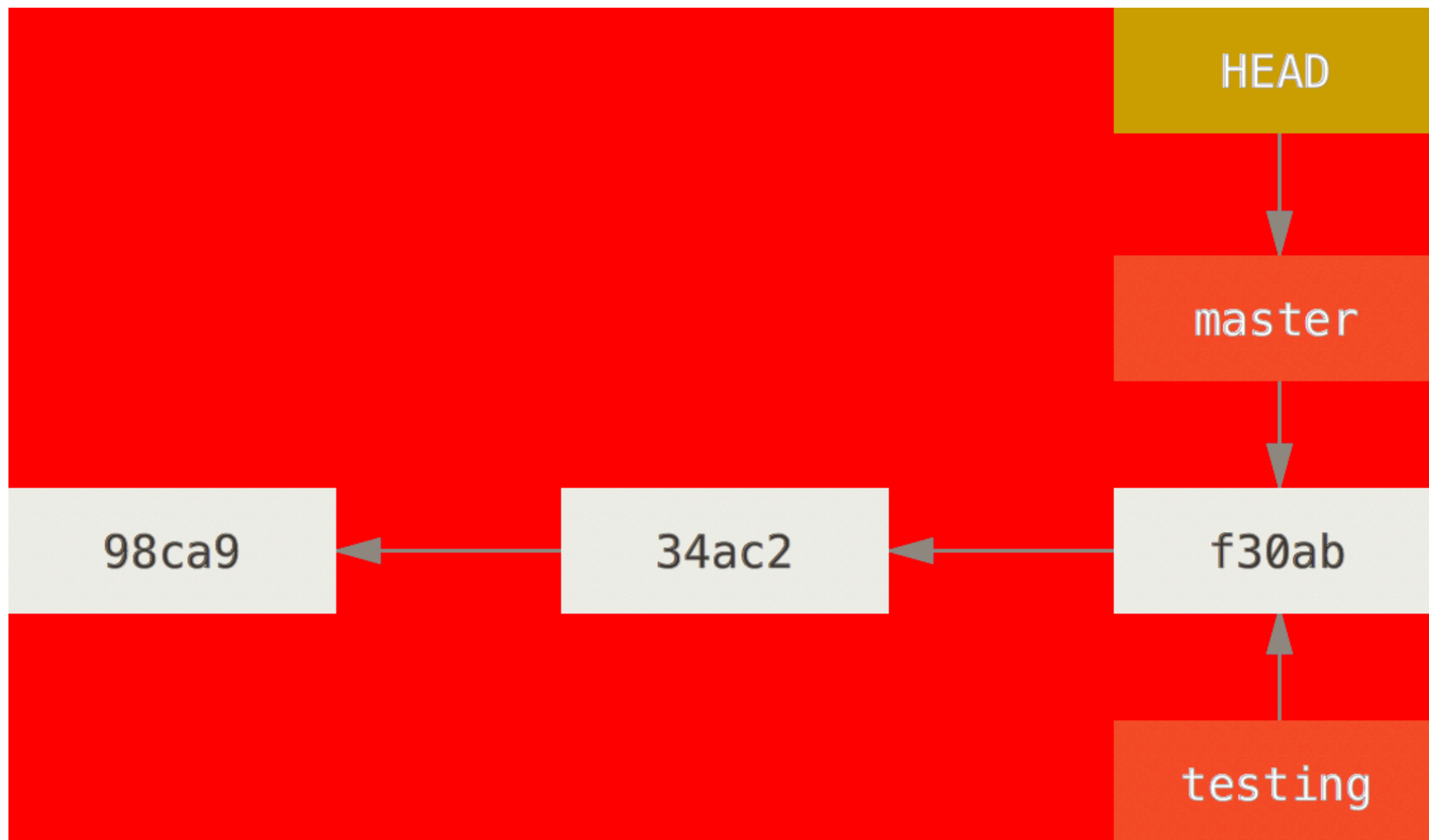
Branches

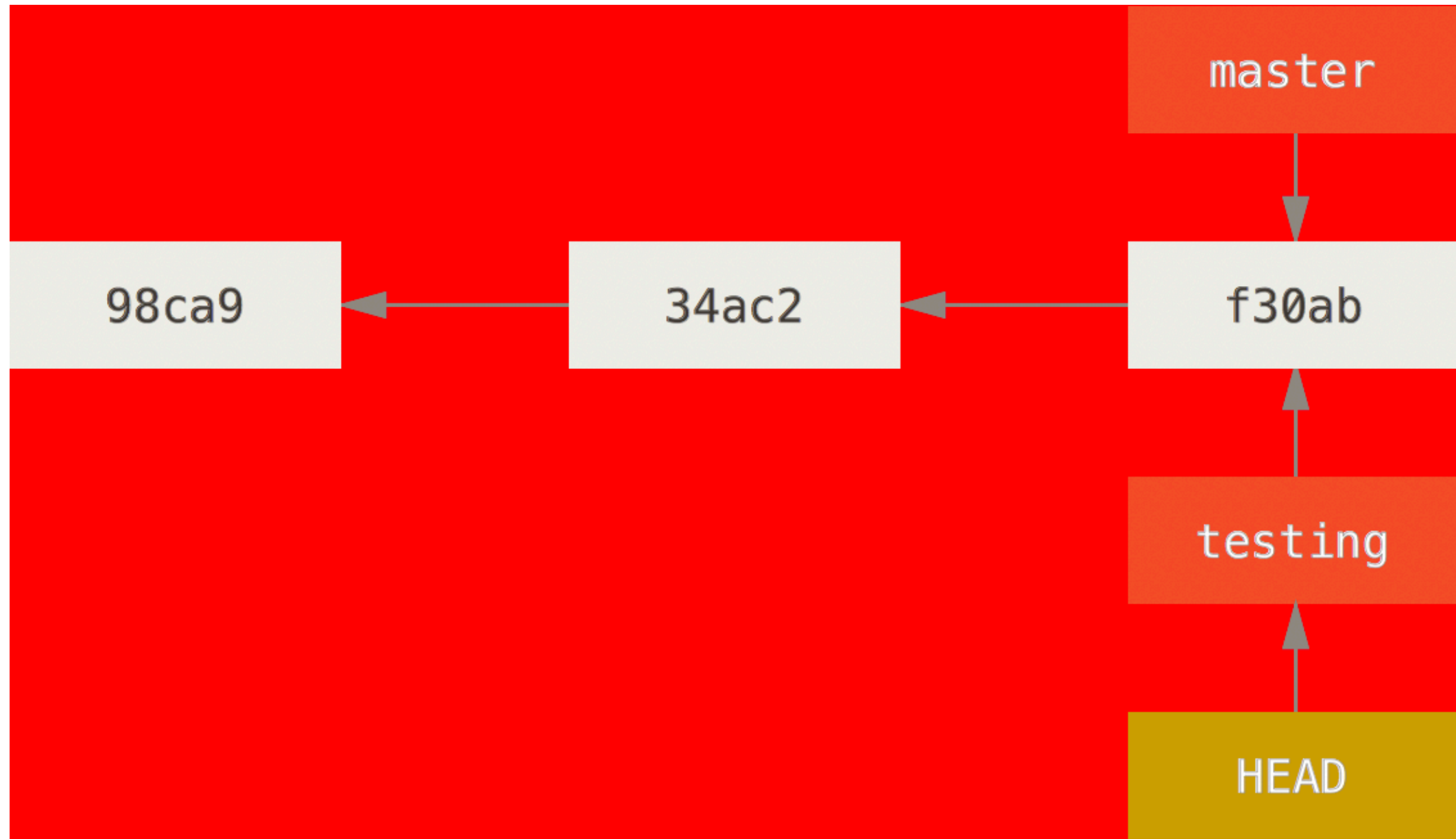


Historique des commits

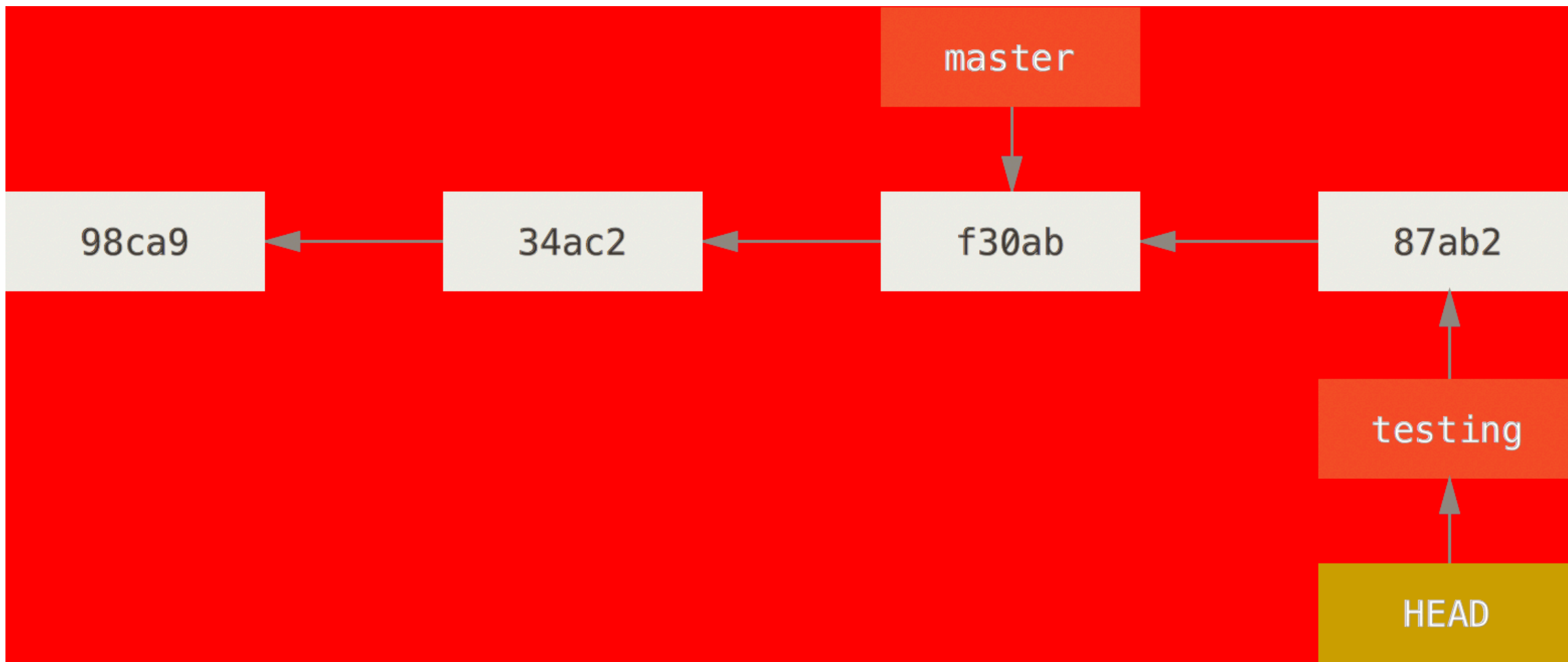


git branch testing

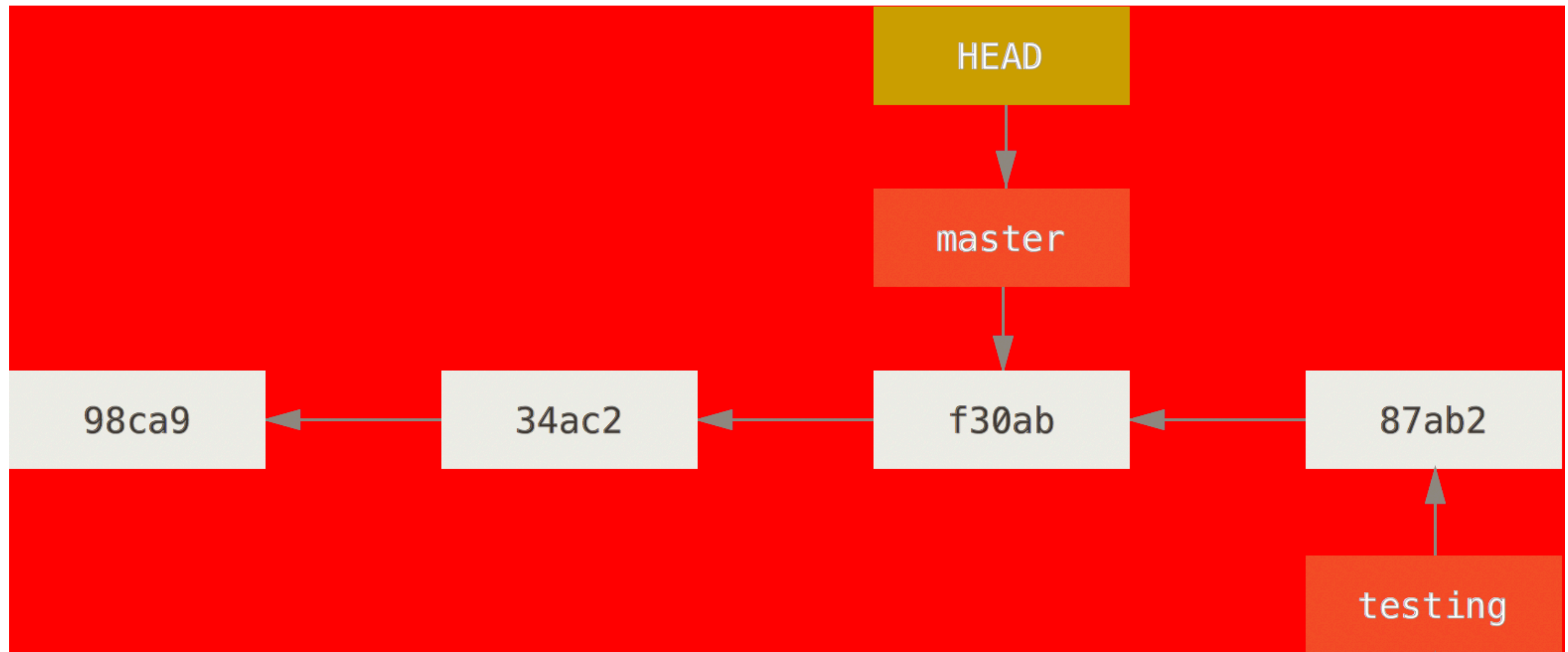




git checkout testing

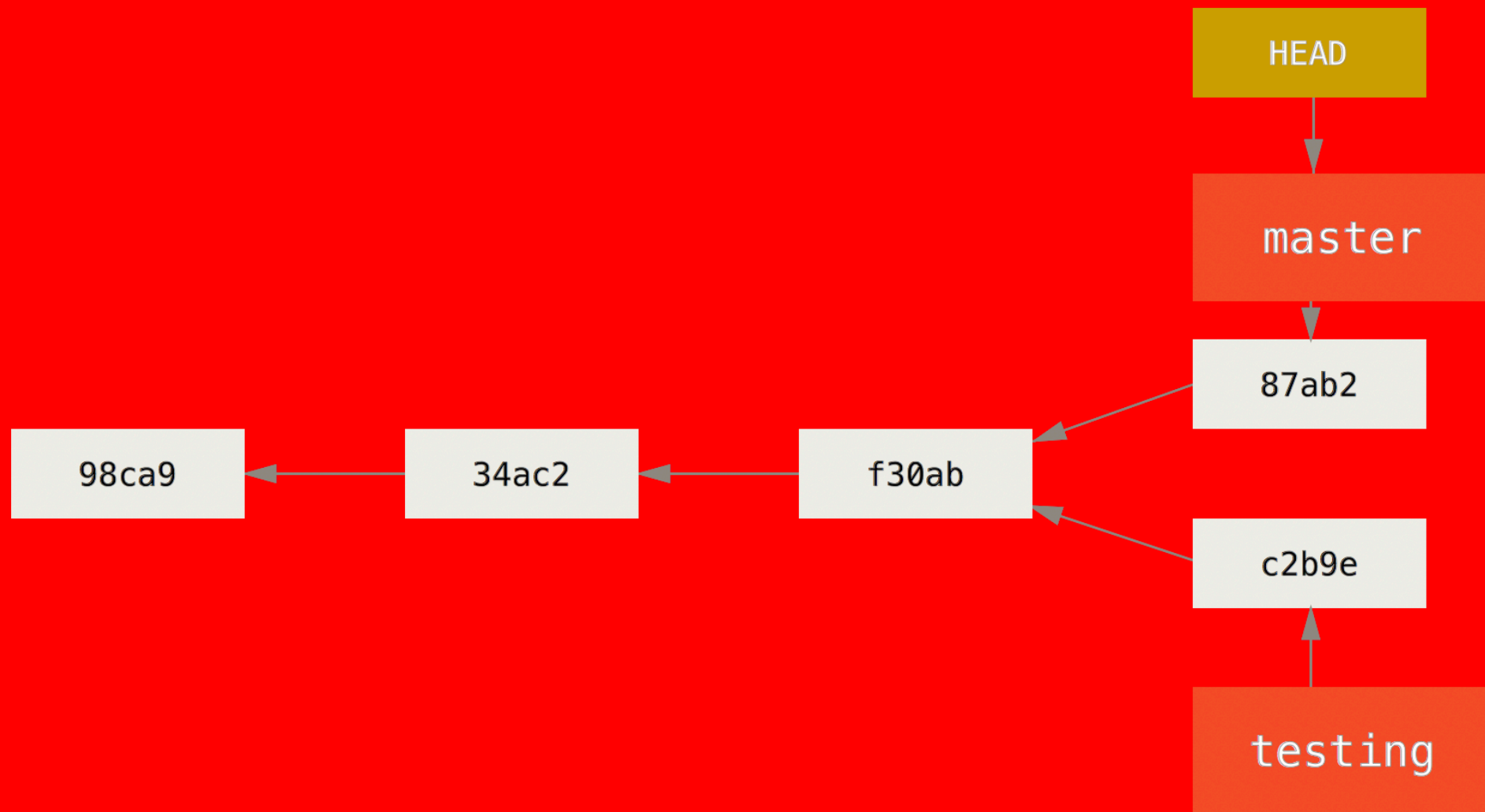


nano readme.md
git commit -a -m “change readme”



git checkout master

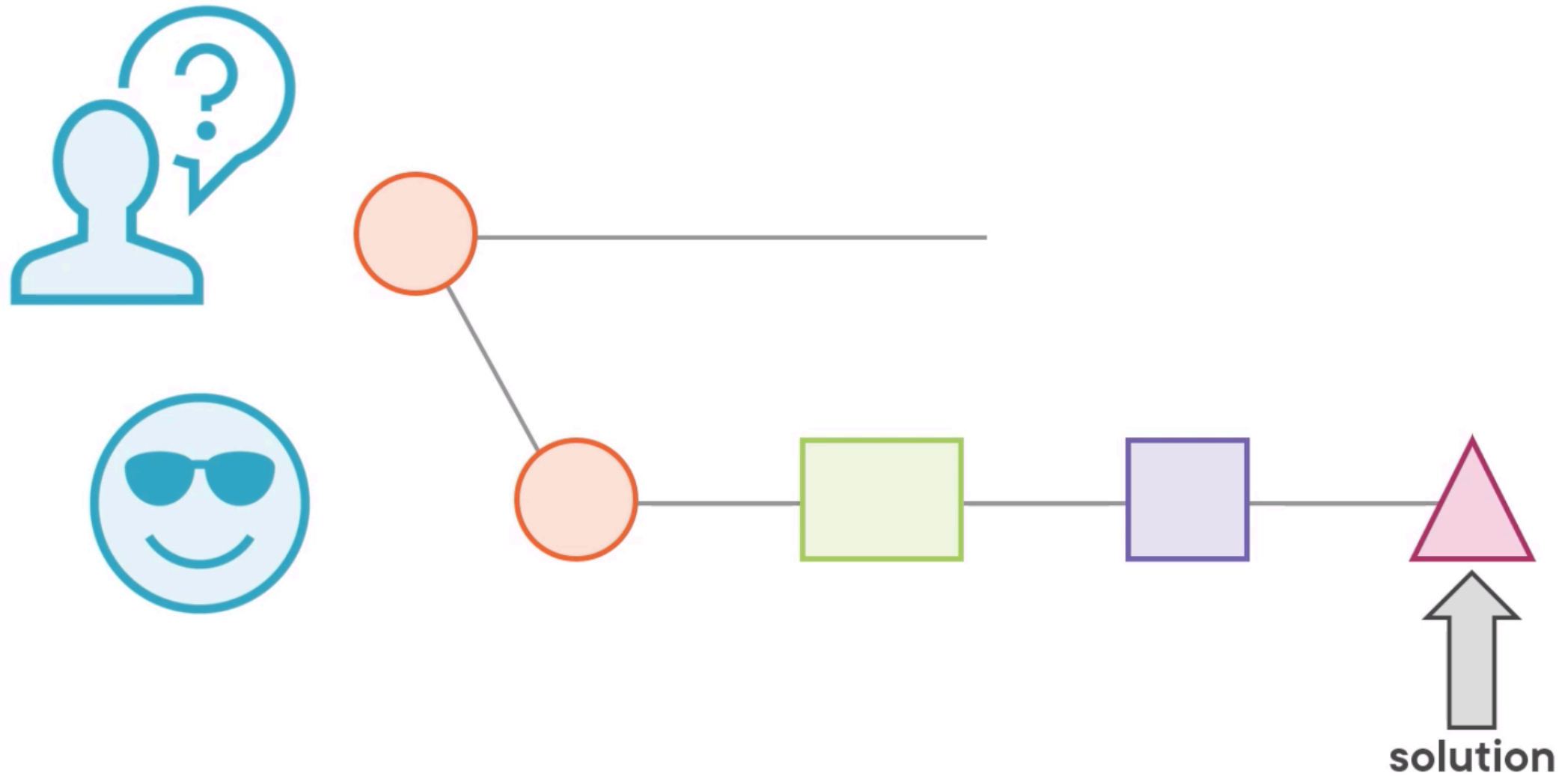
Quand on change de branche, le 'working directory' reflète les changements



nano readme.md
git commit -a -m “autres changements”
git log --oneline --decorate --graph --all

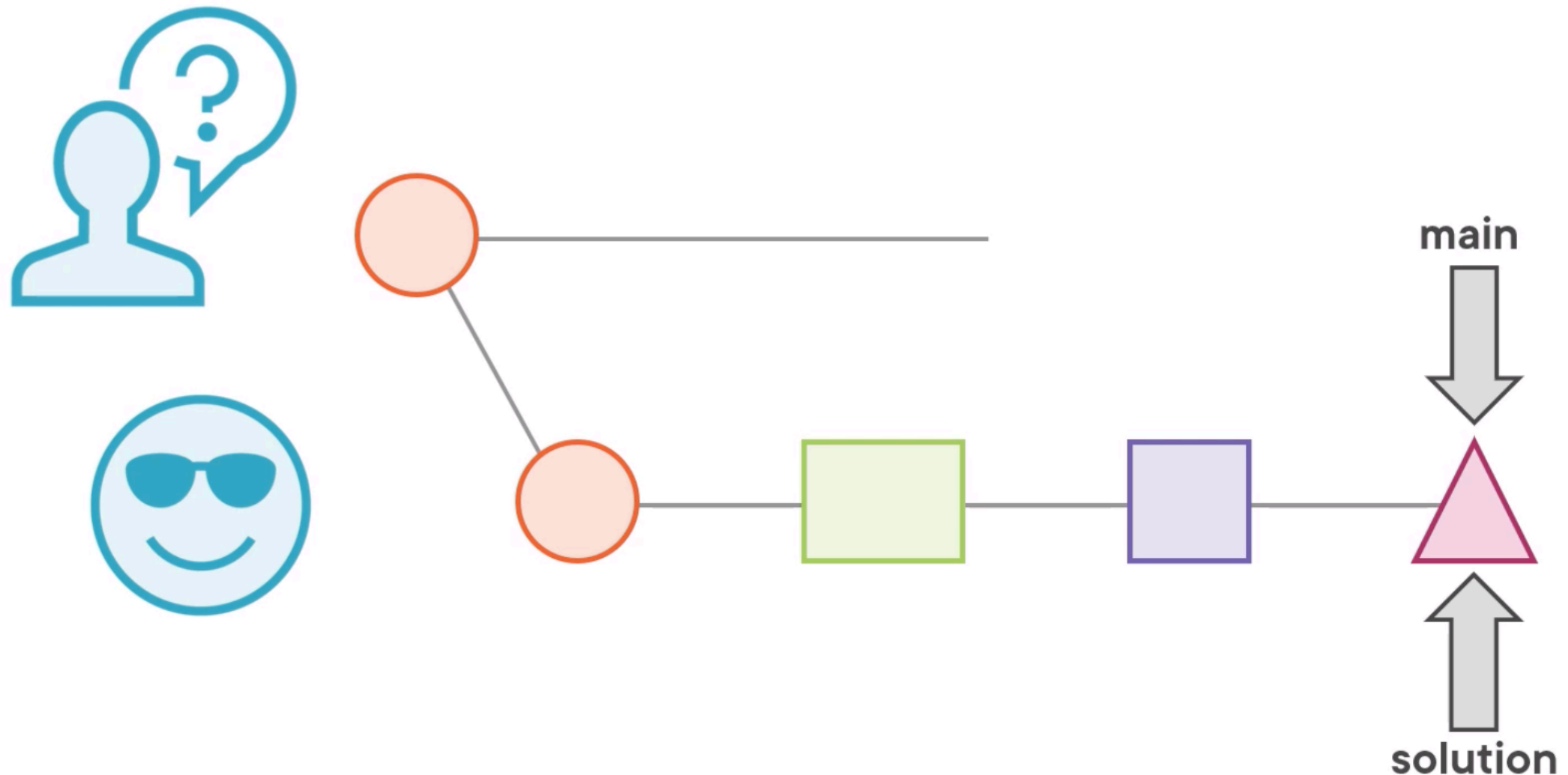
Merge fast-forward

Fast Forward Merge



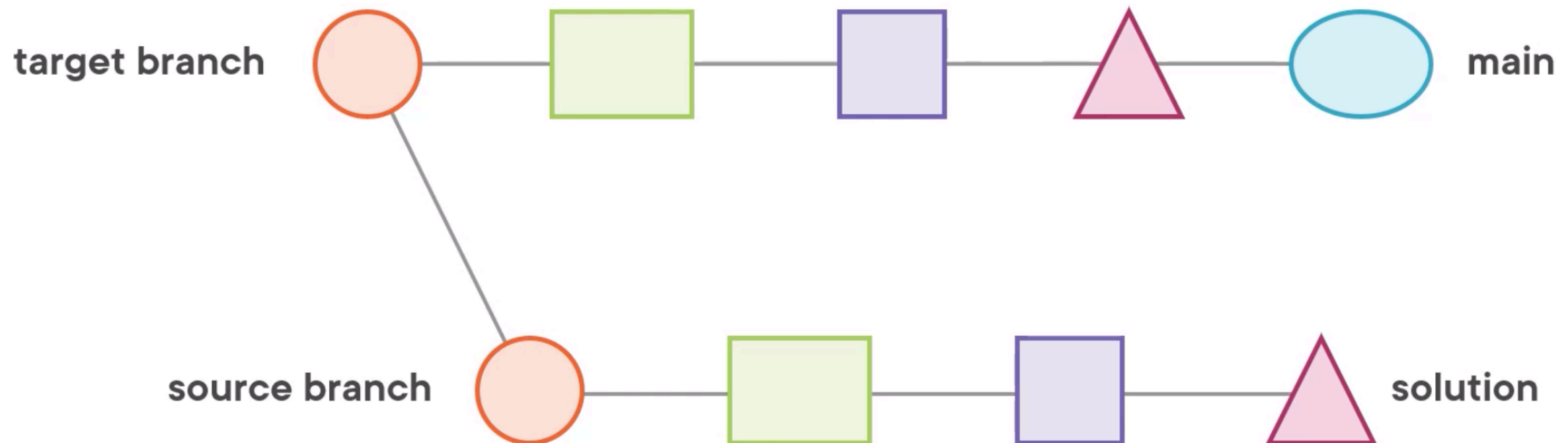
Merge Fast-forward

Fast Forward Merge



Merge

Target and Source



git checkout vs git branch

```
git branch
```

```
git checkout <branch-name>
```

```
git switch <branch-name>
```

```
git checkout -b <branch-name>
```

```
git switch -c <branch-name>
```

Renommer une branche

```
git branch -m <current name> <new name>  
git branch -m quickfix longfix
```

Détruire une branche

```
git branch -d <branchname>  
git branch -d longfix
```

Merge (Simple)

```
git checkout <target-branch>  
git merge <source-branch>  
git checkout main  
git merge ticket1
```


git diff

```
git diff <branch1> <branch2>  
git diff main ticket1
```

```
diff --git a/example.txt b/example.txt
```

```
index 747e2b3..8ef0a69 100644
```

```
--- a/example.txt
```

```
+++ b/example.txt
```


Utilisation de 'remote host' (ex: github)

Utilisation de 'remote host' (ex: github)

- Permet de centraliser les changements de code qui peuvent être partagés avec une équipe
- Permet de créer des branches qui peuvent être également partagées avec une équipe
- Permet d'utiliser les 'Pull Request' (PR), également appelés 'Merge Request'

Exemple

- `git checkout -b newfeature`
- `mkdir newfeature`
- `touch newfeature.txt`
- `git add .`
- `git commit -m "Nouveau feature"`
- `git ls-remote` (la branche n'est pas dans le remote)

Exemple

- `git push -u origin newfeature`
- `git ls-remote` (La branche est maintenant dans le remote)

Exemple

- Pour utiliser la nouvelle branche sur un nouvel ordinateur (ex, d'un autre membre d'une équipe)
- `git fetch origin newfeature`
- `git branch` (ne voit pas la nouvelle branche)
- `git branch -a` (voit la branche du remote)
- `git checkout —track origin/newfeature`

Exemple

```
git push -u origin feature4
```

```
git ls-remote
```

```
git remote -v
```

```
git fetch origin feature4
```

```
git branch -a
```

- ```
git checkout --track origin/feature4
```

# Pull Request

- Feature de Github
- Ne fait pas partie de Git
- Utilisé lorsqu'on termine le travail sur une branche et qu'on veut incorporer celle-ci à 'master' (main, trunk)...
- Facilite la révision de code (code review)
- C'est en fait une requête pour faire un 'merge' d'une branche à une autre. Devrait s'appeler un 'merge request'



# Pour éviter les ‘merge hell’

- Les merge hell se produisent lorsque plusieurs changements conflictent avec d'autres changements ayant eu lieu sur le même (ou les mêmes) fichier.
- S'en suit un travail notable qui consiste à résoudre les conflits avant de pouvoir effectuer un merge
- Une solution simple consiste à faire les ‘merge’ fréquemment de ‘main’ vers la branche dans laquelle on travail.
- De cette façon, si on prend du temps à travailler sur la branche, celle-ci sera plus facile à intégrer à ‘main’ le temps venu.



# Pour éviter les 'merge hell'

- Lorsque tous les conflits ont été résolus (De main vers notre nouvelle branche) on peut alors faire le 'pull request' vers main